

AFRILANG Stdlib

std/c/compbitpack

Français. Bibliothèque AFRILANG 'std/c/compbitpack' (niveau complex, catégorie complex). Elle expose 6 fonction(s) : bitpackEncode, bitpackDecode, bitpackRatio, bitpackSize, bitpackCompressed, bitpackRoundTrip.

```
'import "std/c/compbitpack"' · 'use compbitpack'
```

- 'bitpackEncode(data list number) → list number' - 'bitpackDecode(encoded list number) → list number' - 'bitpackRatio(data list number) → number' - 'bitpackSize(data list number) → number' - 'bitpackCompressed(data list number) → number' - 'bitpackRoundTrip(data list number) → bool'

English. AFRILANG library 'std/c/compbitpack' (tier complex, category complex). Exports 6 function(s): bitpackEncode, bitpackDecode, bitpackRatio, bitpackSize, bitpackCompressed, bitpackRoundTrip.

```
'import "std/c/compbitpack"' · 'use compbitpack'
```

- 'bitpackEncode(data list number) → list number' - 'bitpackDecode(encoded list number) → list number' - 'bitpackRatio(data list number) → number' - 'bitpackSize(data list number) → number' - 'bitpackCompressed(data list number) → number' - 'bitpackRoundTrip(data list number) → bool'

Catégorie / Category	Modules complex (c/)	
Niveau / Tier	complex	June 16, 2026
Fonctions / Functions	6	

Contents

Français

1. Vue d'ensemble

Bibliothèque AFRILANG 'std/c/compbitpack' (niveau complex, catégorie complex). Elle expose 6 fonction(s) : bitpackEncode, bitpackDecode, bitpackRatio, bitpackSize, bitpackCompressed, bitpackRoundTrip.

```
'import "std/c/compbitpack"' · 'use compbitpack'
```

```
- `bitpackEncode(data list number) → list number`
- `bitpackDecode(encoded list number) → list number`
- `bitpackRatio(data list number) → number`
- `bitpackSize(data list number) → number`
- `bitpackCompressed(data list number) → number`
- `bitpackRoundTrip(data list number) → bool`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/compbitpack"
use compbitpack
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés – graphes, NLP, réseaux complexes.

3. Référence API

- bitpackEncode(data list number) → list•
bitpackDecode(encoded list number) → list•
bitpackRatio(data list number) → number•
bitpackSize(data list number) → number•
bitpackCompressed(data list number) → number•
bitpackRoundTrip(data list number) → bool

4. Exemples d'utilisation

```
import "std/c/compbitpack"
use compbitpack
```

```
create result = bitpackEncode(/* args */)
say result
```

```
create result = bitpackDecode(/* args */)
say result
```

```
create result = bitpackRatio(/* args */)
say result
```

```
create result = bitpackSize(/* args */)
say result
```

```
create result = bitpackCompressed(/* args */)
say result
```

```
create result = bitpackRoundTrip(/* args */)
say result
```

5. Bonnes pratiques

- Préférez ``import "std/c/compbitpack"`` en tête de fichier.
- Lancez ``afriLang check`` avant de livrer.
- Combinez avec les modules stdlib de la même catégorie.
- Voir `/stdlib/` pour le source live et les démos playground.
- Signalez les problèmes sur GitHub si une export manque.

6. Annexe — source

module compbitpack

```
function bitpackEncode(data list number) returns list number  
end
```

```
function bitpackDecode(encoded list number) returns list number  
end
```

```
function bitpackRatio(data list number) returns number  
end
```

```
function bitpackSize(data list number) returns number  
end
```

```
function bitpackCompressed(data list number) returns number  
end
```

```
function bitpackRoundTrip(data list number) returns bool  
end  
end
```

English

1. Overview

AFRILANG library 'std/c/compbitpack' (tier complex, category complex). Exports 6 function(s): bitpackEncode, bitpackDecode, bitpackRatio, bitpackSize, bitpackCompressed, bitpackRoundTrip.

```
'import "std/c/compbitpack"' · 'use compbitpack'
```

```
- `bitpackEncode(data list number) → list number`
- `bitpackDecode(encoded list number) → list number`
- `bitpackRatio(data list number) → number`
- `bitpackSize(data list number) → number`
- `bitpackCompressed(data list number) → number`
- `bitpackRoundTrip(data list number) → bool`
```

2. Installation & import

Add the module to your program:

```
import "std/c/compbitpack"
use compbitpack
```

Tier: complex · Category: Complex modules (c/)
Advanced domains – graphs, NLP, complex networks.

3. API reference

- bitpackEncode(data list number) → list•
- bitpackDecode(encoded list number) → list•
- bitpackRatio(data list number) → number•
- bitpackSize(data list number) → number•
- bitpackCompressed(data list number) → number•
- bitpackRoundTrip(data list number) → bool

4. Usage examples

```
import "std/c/compbitpack"
use compbitpack
```

```
create result = bitpackEncode(/* args */)
say result
```

```
create result = bitpackDecode(/* args */)
say result
```

```
create result = bitpackRatio(/* args */)
say result
```

```
create result = bitpackSize(/* args */)
say result
```

```
create result = bitpackCompressed(/* args */)
say result
```

```
create result = bitpackRoundTrip(/* args */)
say result
```

5. Best practices

- Prefer ``import "std/c/compbitpack"`` at the top of your file.
- Run ``afrilang check`` before shipping.
- Combine with related stdlib modules from the same category.
- See `/stdlib/` for the live source and playground demos.
- Report issues on GitHub if an export is missing or undocumented.

6. Appendix — source

module compbitpack

```
function bitpackEncode(data list number) returns list number
end
```

```
function bitpackDecode(encoded list number) returns list number
end
```

```
function bitpackRatio(data list number) returns number
end
```

```
function bitpackSize(data list number) returns number
end
```

```
function bitpackCompressed(data list number) returns number
end
```

```
function bitpackRoundTrip(data list number) returns bool
end
end
```

Référence rapide / Quick reference

- Import : `import "std/c/compbitpack"`
- Vérification : `afrilang check`
- Documentation : <https://afrilang.dev/stdlib/std/c/compbitpack/>

Source (extrait)

```
module compbitpack

function bitpackEncode(data list number) returns list number
end

function bitpackDecode(encoded list number) returns list number
end

function bitpackRatio(data list number) returns number
end

function bitpackSize(data list number) returns number
end

function bitpackCompressed(data list number) returns number
```

```
end
function bitpackRoundTrip(data list number) returns bool
end
end
```